About this PDF

Built only from: **Theory of Computation (Computability and Complexity)** — https://nptel.ac.in/courses/106104028

Licence: CC BY-SA 4.0. Accessed .

Covers 8 of 8 syllabus sub-topics of this unit.

Unsolvable Problems and Computable Functions

Definition Computable functions are mappings from strings to strings that can be produced by a deterministic Turing machine that halts on all inputs; if the machine halts on all inputs the function is total recursive, otherwise it is partial recursive. A computational decision problem is decidable if there exists an algorithm that halts and correctly decides membership for every input; if no such algorithm exists the problem is undecidable. A Universal Turing Machine (UTM) can simulate any Turing machine on any input, demonstrating general-purpose programmable computing. The Halting Problem asks whether an arbitrary Turing machine M halts on a given input string w, and it was proved undecidable by Turing using a diagonalization argument.

Explanation The existence of a UTM shows that a single machine can interpret and execute any other machine's program, establishing the concept of general-purpose computing. The Halting Problem's undecidability follows from a diagonalization argument that constructs a machine that behaves opposite to the assumed halting decider, leading to a contradiction. Thus, no algorithm can correctly decide halting for all possible machine-input pairs.

Example The Halting Problem is the concrete example of an undecidable problem: given a pair (M, w), determine whether M halts on input w.

- A computable function is produced by a deterministic Turing machine that halts on all inputs.
- Total recursive functions halt on every input; partial recursive functions may not halt on some inputs.
- A decision problem is decidable if an algorithm halts for every input; otherwise it is undecidable.
- A Universal Turing Machine can simulate any other Turing machine on any input.
- The Halting Problem is undecidable, proven by a diagonalization argument.

PCP-MPCP

Definition

The Post Correspondence Problem (PCP) is a combinatorial problem over strings. Given two lists of strings of equal length over an alphabet Sigma, List A = (w1, w2, …, w_k) and List B = (x1, x2, …, x_k), one asks whether there exists a non-empty sequence of indices (i1, i2, …, i_m) such that the concatenation of the strings from the first list equals the concatenation of the strings from the second list: w_i1 w_i2 … w_i_m = x_i1 x_i2 … x_i_m. Such a sequence is called a match or correspondence. In the Modified Post Correspondence Problem (MPCP), the match must begin with the first pair of strings, meaning i1 = 1. PCP is undecidable for alphabets with two or more symbols, and MPCP is also undecidable and serves as an intermediate reduction step to prove the undecidability of standard PCP, context-free grammar ambiguity, and other formal language properties.

Explanation

PCP is undecidable, meaning that no algorithm can decide membership for all instances of the problem. The undecidability of PCP for alphabets with two or more symbols is a fundamental result in computability theory. MPCP, which imposes the additional constraint that the first index must be 1, is likewise undecidable. MPCP is used as an intermediate reduction step in proofs of undecidability for other problems, such as the ambiguity of context-free grammars.

Key points

- PCP asks for a non-empty sequence of indices that makes the concatenations of two lists of strings equal; such a sequence is called a match or correspondence.
- PCP is undecidable for alphabets with two or more symbols.
- In MPCP the match must start with the first pair of strings (i1 = 1); MPCP is also undecidable and is used as an intermediate reduction step in proving other undecidability results.

Recursive and recursively enumerable languages

Definition

A language L is **Recursive (R)** if there exists a total Turing machine M that halts on every input w, accepting if w in L and rejecting if w not in L. Membership in a recursive language is fully decidable by an algorithm. A language L is **Recursively Enumerable (RE)** if there exists a Turing machine M such that for every w in L, M halts and accepts. If w not in L, M may either halt and reject or loop infinitely.

Explanation

Recursive languages are closed under union, intersection, complementation, concatenation, and Kleene star. Recursively enumerable languages are closed under union, intersection, concatenation, and Kleene star, but are not closed under complementation. A language L is Recursive if and only if both L and its complement complement-of-L are Recursively Enumerable.

Key points

- Recursive languages are decidable by a total Turing machine that halts on all inputs.
- Recursively enumerable languages are recognizable by a Turing machine that may not halt on non-members.
- Recursive languages are closed under union, intersection, complementation, concatenation, and Kleene star.
- Recursively enumerable languages are closed under union, intersection, concatenation, and Kleene star, but not under complementation.
- A language is recursive iff both it and its complement are recursively enumerable.

Properties

Definition

Recursive languages are those for which there exists a total Turing machine that halts on every input, accepting if the input is in the language and rejecting otherwise. Recursively enumerable languages are those for which there exists a Turing machine that halts and accepts for every string in the language, but may not halt for strings not in the language.

Explanation

Recursive languages are closed under union, intersection, complementation, concatenation, and Kleene star. Recursively enumerable languages are closed under union, intersection, concatenation, and Kleene star, but are not closed under complementation. Moreover, a language is recursive if and only if both it and its complement are recursively enumerable.

Key points

- Recursive languages are closed under union, intersection, complementation, concatenation, and Kleene star.
- Recursively enumerable languages are closed under union, intersection, concatenation, and Kleene star, but not under complementation.
- A language is recursive iff both it and its complement are recursively enumerable.
- PCP and MPCP are undecidable for alphabets with two or more symbols.
- MPCP is also undecidable and serves as an intermediate reduction step to prove undecidability of standard PCP, context-free grammar ambiguity, and other formal language properties.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Universal Turing machine

Definition

A Universal Turing Machine U is an interpretive machine capable of simulating any arbitrary Turing machine M operating on an arbitrary input string w. The pair (M, w) is encoded into a standardized binary representation <M, w>. The UTM reads this encoding and simulates M step by step. The existence of UTM demonstrates general-purpose programmable computing.

Explanation

The UTM interprets the encoded description of another machine and its input, then reproduces the computation of that machine step by step. This ability to simulate any Turing machine shows that a single machine can perform any computable function, making it a general-purpose programmable computer.

Algorithm / Steps

1. Read the binary encoding <M, w> that represents the machine M and its input w.
2. Simulate the operation of M on w, performing each transition of M one step at a time.

Key points

- A UTM can simulate any Turing machine given its encoded description and input.
- The encoding <M, w> is a standardized binary representation.
- The UTM reads this encoding and then simulates M step by step.
- Existence of a UTM shows that general-purpose programmable computing is possible.

Tractable and Intractable problems P and NP completeness

Definition

- A problem is tractable if it can be solved by an algorithm running in polynomial time O(n^k) for input size n. Problems requiring super-polynomial or exponential time O(2^n) are deemed intractable.
- Class P consists of all decision problems solvable by a deterministic Turing machine in polynomial time.
- Class NP consists of all decision problems verifiable in polynomial time, or equivalently, solvable by a non-deterministic Turing machine in polynomial time.
- A problem X is NP-complete if (1) X is in NP, and (2) every problem in NP is polynomial-time reducible to X (NP-hard).

Explanation

- Polynomial-time algorithms are considered efficient and feasible for practical use, whereas exponential-time algorithms grow too fast to be practical for large inputs.

- The equivalence between verification in polynomial time and non-deterministic polynomial-time solvability defines the class NP.
- NP-completeness captures the hardest problems in NP: if any NP-complete problem can be solved in polynomial time, then every problem in NP can be solved in polynomial time.

Example

- The Travelling Salesman Problem (TSP) decision version asks whether there exists a tour visiting each city exactly once with total distance <= k; TSP is a classic NP-complete problem.
- Kruskal's algorithm finds a Minimum Spanning Tree (MST) in a connected, undirected, edge-weighted graph using a greedy polynomial-time approach running in O(E log E) or O(E log V) time; this problem belongs to class P and is computationally tractable.

Key points

- Tractable problems are solvable in polynomial time; intractable problems require super-polynomial or exponential time.
- Class P contains problems solvable deterministically in polynomial time; class NP contains problems verifiable in polynomial time.
- A problem is NP-complete if it is in NP and every NP problem reduces to it in polynomial time.
- TSP is an example of an NP-complete problem, while finding an MST via Kruskal's algorithm is an example of a tractable problem in P.

Kruskal’s algorithm

Definition

"Kruskal's algorithm is a greedy polynomial-time algorithm that finds a minimum spanning tree in a connected, undirected, edge-weighted graph."

Explanation

"Unlike NP-complete problems, finding an MST via Kruskal's algorithm belongs to class P and is computationally tractable."

Key points

- Greedy polynomial-time algorithm.
- Runs in O(E log E) or O(E log V) time.
- Finds a minimum spanning tree in a connected, undirected, edge-weighted graph.
- Belongs to class P and is computationally tractable.

Travelling Salesman Problem- 3-CNF SAT problems.

Definition

The Travelling Salesman Problem (TSP) is a decision problem that asks whether there exists a tour visiting each city exactly once with total distance <= k. The 3-CNF SAT problem asks whether a Boolean formula in conjunctive normal form, where each clause has exactly three literals, has a satisfying truth assignment.

Explanation

Both TSP and 3-CNF SAT are NP-complete. The Cook-Levin theorem proved that SAT is NP-complete, and 3-CNF SAT is a restricted form that remains NP-complete, serving as a baseline reduction problem for proving other problems NP-hard. TSP is also a classic NP-complete problem, meaning every problem in NP can be polynomial-time reduced to it.

Key points

- TSP asks for a tour of total distance <= k.
- 3-CNF SAT asks for a satisfying assignment of a formula with clauses of exactly three literals.
- Both problems are NP-complete; 3-CNF SAT is the baseline reduction problem.
- Kruskal's algorithm finds a minimum spanning tree in polynomial time, contrasting with the NP-complete nature of TSP.

Source and licence

This PDF is adapted from **Theory of Computation (Computability and Complexity)** (https://nptel.ac.in/courses/106104028), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).